UDC-WQIS Azure Migration Report

Complete Infrastructure Migration — Vercel to Microsoft Azure

Prepared by: Oli T. Oli (Co-Founder & CTO, DAPS Analytics) **In partnership with:** Dr. Tolessa Deksissa (UDC WRRI) **Date:** April 18, 2026
Duration: Single session (~4 hours) **Status:** Phases 1–5 Complete, Phase 6 (DNS Cutover) Pending

---

Executive Summary

The UDC Water Quality Information System (UDC-WQIS) has been successfully migrated from Vercel (free Hobby plan) to Microsoft Azure production-grade infrastructure. The migration was completed in a single working session with zero data loss and zero downtime. The platform now runs on Azure Container Apps with a dedicated PostgreSQL database, automated data ingestion via Container Apps Jobs, centralized secret management, comprehensive monitoring, and a fully automated CI/CD pipeline.

Before: A free-tier Vercel deployment with a single cron job, 60-second function timeout, no SLA, and limited scalability.

After: A production-ready Azure deployment with 3 automated cron jobs, no timeout constraints, government-grade infrastructure, and a clear path to FedRAMP compliance.

Migration Phases Completed

Phase 1: Database Migration

Neon PostgreSQL (Serverless) → Azure Database for PostgreSQL (Flexible Server)

Metric	Detail
Source	Neon PostgreSQL 17.8 (AWS us-east-1)
Target	Azure PostgreSQL 16.13 (Central US)
Server	`udc-wqis-db.postgres.database.azure.com`
SKU	Burstable B1ms (1 vCore, 2GB RAM, 32GB storage)
Migration method	Custom Node.js script (`scripts/migrate-to-azure.mjs`)
Data transferred	206,537 rows across 5 tables
Data loss	Zero
Downtime	Zero (parallel databases during migration)

Data verification:

Table	Rows Migrated	Status
stations	12	Verified
parameters	25	Verified
readings	58,351	Verified
measurements	148,019	Verified (later grew to 155,339 via live ingestion)
ingestion_log	30	Verified (now 31 after Azure cron test)

Code change: Updated `src/lib/db.ts` to auto-detect database provider — Neon URLs (`.neon.tech`) use the WebSocket-based Neon driver; all other PostgreSQL URLs use the standard `pg` driver. This makes the codebase portable across any PostgreSQL provider.

Phase 2: Application Deployment

Vercel Serverless → Azure Container Apps

Metric	Detail
Container Registry	`udcwqisacr.azurecr.io` (Basic tier)
Container App	`udc-wqis` (Consumption plan)
Resources	0.5 vCPU, 1 GB RAM
Min replicas	1 (prevents cold starts)
Max replicas	3 (auto-scales under load)
Image	Multi-stage Docker build (Node 20 Alpine, standalone Next.js)
Health endpoint	`/api/health` — returns status, DB connectivity, station count, uptime
Production URL	https://udc-wqis.happycoast-d9b0bcde.centralus.azurecontainerapps.io/

Code changes:

- `next.config.ts` : Added `output: "standalone"` for Docker compatibility
- `package.json` : Added `pg` dependency for standard PostgreSQL connections
- `Dockerfile` : Already existed and required no modifications

Verified endpoints (all 14):

Endpoint	Status
`/api/health`	`{"status":"healthy","database":{"status":"connected","stations":12}}`
`/api/stations`	Returns all 12 stations with readings
`/api/stations/[id]/history`	Station time-series data
`/api/measurements`	EAV parameter queries
`/api/parameters`	25 parameter definitions
`/api/export`	CSV/JSON data export
`/api/ingest`	USGS/EPA/WQP ingestion (API key protected)
`/api/ingestion-log`	Ingestion run history
`/api/chat`	AI research assistant (Claude)
`/api/admin/*`	Station/readings CRUD, upload, AI column mapping

Phase 3: Automated Data Ingestion

Vercel Cron (1 job limit) → Azure Container Apps Jobs (unlimited)

Job	Schedule	Source	Avg Records/Run

`usgs-ingest-cron`	Daily 06:00 UTC	USGS NWIS real-time sensors	~9,120
`epa-ingest-cron`	Daily 07:00 UTC	EPA Water Quality Portal	TBD (first scheduled run pending)
`wqp-ingest-cron`	Monday 08:00 UTC	Water Quality Portal (broad)	TBD (first scheduled run pending)

Architecture: Each cron job is a lightweight Node.js container (`cron-runner.mjs` , 2 files, <2KB) that calls the app's `/api/ingest` endpoint with the appropriate source parameter and API key authentication.

Verification: Manual trigger of `usgs-ingest-cron` completed successfully:

- Status: `Succeeded`
- Records ingested: 11,214
- Logged in ingestion_log as entry #31
- Data visible in dashboard immediately

Improvement over Vercel:

Capability	Vercel (Before)	Azure (After)
Cron jobs allowed	1 (Hobby plan limit)	Unlimited
Function timeout	60 seconds	120 seconds (configurable)
Retry on failure	None	Automatic (1 retry)
Execution logs	Limited	Full container logs + App Insights
Manual trigger	Admin panel only	CLI + Admin panel

Phase 4: Secrets Management & Monitoring

Plaintext env vars → Azure Key Vault + Azure Monitor

Key Vault (`udc-wqis-kv`)

Secret	Purpose
`database-url`	Azure PostgreSQL connection string
`anthropic-api-key`	Claude AI research assistant
`admin-api-key`	Admin panel authentication
`ingest-api-key`	Ingestion endpoint authentication

Monitoring Stack

Component	Configuration
Application Insights (`udc-wqis-monitor`)	Telemetry and diagnostics
Availability Test (`Health Check Ping`)	Pings `/api/health` every 5 minutes from 3 US locations (Virginia, Chicago, San Jose)
Alert: Container Restarts (Severity 1)	Fires if >3 restarts in 15 minutes
Alert: Database CPU (Severity 2)	Fires if CPU >80% sustained for 15 minutes
Alert: Database Storage (Severity 2)	Fires if storage usage >80%
Action Group (`udc-wqis-alerts`)	Email notifications on alert trigger

Phase 5: CI/CD Pipeline

Manual deploys → Automated GitHub Actions

Component	Detail
Workflow file	`.github/workflows/deploy-azure.yml`
Trigger	Push to `main` branch
Service Principal	`udc-wqis-github-deploy`
Roles	`Contributor` (resource group) + `AcrPush` (container registry)

Pipeline stages:

```
git push origin main
    │
    ▼
┌──────────────────────┐
│  Stage 1: TEST       │
│  • npm ci            │
│  • npm run db:seed   │
│  • npm test (52 tests)│
│  ❌ Fails → deploy blocked│
└──────────┬───────────┘
           │ ✅ Pass
           ▼
┌──────────────────────┐
│  Stage 2: DEPLOY     │
│  • Azure login       │
│  • Docker build      │
│  • Push to ACR       │
│  • Update Container App│
└──────────────────────┘
```

GitHub Secrets configured:

Secret	Purpose
`AZURE_CREDENTIALS`	Service principal JSON (clientId, clientSecret, subscriptionId, tenantId)
`ACR_LOGIN_SERVER`	`udcwqisacr.azurecr.io`
`ACR_USERNAME`	`udcwqisacr`
`ACR_PASSWORD`	Container registry push credential

Phase 6: DNS Cutover (Pending)

Status: Deferred — running Vercel and Azure in parallel for testing period.

Both platforms auto-deploy on push to `main`:

- **Vercel:** https://udc-one.vercel.app (Neon PostgreSQL)
- **Azure:** https://udc-wqis.happycoast-d9b0bcde.centralus.azurecontainerapps.io/ (Azure PostgreSQL)

When ready:

1. Purchase custom domain (e.g., `udcwqis.org` — confirmed available)
2. Configure CNAME + managed TLS on Azure Container Apps
3. Update cron job `APP_URL` env var
4. Decommission Vercel project
5. Decommission Neon PostgreSQL database

Azure Resource Inventory

Resource	Name	Type	Location
Resource Group	`rg-udc-wqis-cu`	—	Central US
PostgreSQL Server	`udc-wqis-db`	Burstable B1ms	Central US
Container App	`udc-wqis`	Consumption (0.5 vCPU, 1GB)	Central US
Container Registry	`udcwqisacr`	Basic	Central US
Key Vault	`udc-wqis-kv`	Standard	Central US
Container Apps Job	`usgs-ingest-cron`	Schedule (daily 06:00 UTC)	Central US
Container Apps Job	`epa-ingest-cron`	Schedule (daily 07:00 UTC)	Central US
Container Apps Job	`wqp-ingest-cron`	Schedule (weekly Mon 08:00 UTC)	Central US
Application Insights	`udc-wqis-monitor`	—	Central US
Storage Account	`udcwqisfuncstor`	Standard LRS	Central US
Log Analytics Workspace	`workspace-rgudcwqiscuuAFB`	Free tier	Central US

Monthly Cost

Service	Cost
Azure PostgreSQL (Burstable B1ms)	$12.80
Container Apps (0.5 vCPU, 1GB, min-replicas=1)	$14.60
Container Registry (Basic)	$5.00
Container Apps Jobs (3 cron jobs)	$0.10
Key Vault (4 secrets)	$0.02
Storage Account	$0.02
Application Insights	Free tier
Log Analytics	Free tier (5GB/mo)
Monthly Total	**$32.54**
Annual Total	**$390.48**

Code Changes Summary

File	Change	Lines

`src/lib/db.ts`	Auto-detect Neon vs standard `pg` driver	~30 lines modified
`next.config.ts`	Added `output: "standalone"`	1 line
`package.json`	Added `pg` dependency	1 line
`.github/workflows/deploy-azure.yml`	New CI/CD workflow	40 lines (new file)
`scripts/migrate-to-azure.mjs`	Database migration script	120 lines (new file)
`azure-functions/`	Cron runner container	3 files (new)

Total impact: ~50 lines modified in existing code, ~165 lines in new utility files. Zero changes to any UI component, API route logic, or business logic.

Documents Produced

Document	Purpose	Location
Azure Migration Plan	6-phase technical migration plan	`docs/AZURE_MIGRATION_PLAN.pdf`
Startup Application	Microsoft Founders Hub credit application	`docs/AZURE_STARTUP_APPLICATION.pdf`
Grant Use Cases	EPA, NSF, USDA, DC DOEE grant narratives	`docs/GRANT_USE_CASES.pdf`
Cost Analysis	Infrastructure economics and scaling model	`docs/AZURE_COST_ANALYSIS.pdf`
Migration Report	This document	`docs/AZURE_MIGRATION_REPORT.pdf`

Risk Assessment

Risk	Mitigation	Status
Data loss during migration	Full row-count verification across all tables	Verified zero loss
Ingestion downtime	Vercel cron still active during parallel period	Active
Neon driver incompatibility with Azure PG	Auto-detect driver based on connection URL	Resolved
Cold start latency	Set min-replicas=1	Active
Secret exposure	All credentials stored in Azure Key Vault	Complete
Deployment failures	CI/CD runs 52 tests before deploy; failures block release	Active
Database growth	46MB current; 32GB limit not reached for ~6 years at current rate	Monitored

---

Next Steps

1. **Parallel testing period** — Run Vercel and Azure side by side for several days, comparing data freshness and API responses
2. **Custom domain** — Purchase `udcwqis.org` (confirmed available, ~$12/yr) and configure on Azure
3. **DNS cutover** — Point domain to Azure, decommission Vercel and Neon
4. **Azure for Startups** — Apply for Microsoft Founders Hub credits ($5K–$150K)
5. **Grant applications** — Submit EPA EJCPS and NSF S&CC proposals using prepared narratives
6. **White-label scaling** — Deploy additional Container Apps for new watershed clients

---

UDC-WQIS is a product of DAPS Analytics (Data Analytics for Practical Solutions), developed by Oli T. Oli in partnership with Dr. Tolessa Deksissa and UDC's Water Resources Research Institute.